Ex 13: Limitations of Low-Code Platforms in Complex Enterprise Systems

Case Study: Patient Appointment Management at Hospital-Network Scale

V Vijay Prakash
241401128

The hospital network, its requirements and its outcome figures in this report are hypothetical. They are used to analyse where low-code struggles at scale and are not measurements from a real organisation.

1. Aim

To study the limitations of low-code platforms when they are used for large, complex enterprise systems, using an illustrative case that extends the Patient Appointment Booking application built in this lab series, and to suggest how low-code can be used sensibly alongside conventional engineering.

2. Introduction

Low-code platforms let teams build applications quickly through visual design, ready-made components and configuration. That speed is attractive, and it worked well for the booking form, workflows, reports and dashboards built in the earlier experiments. A core enterprise system is a different matter: it may need very high transaction volumes, many business rules, links to many other systems, strict compliance and a service life of many years. This report looks at where those demands expose the limits of low-code.

241401128

3. Background: What Low-Code Provides

A low-code platform offers a visual builder with forms, reports, workflow tools, connectors and templates, with a scripting layer (such as Deluge in Zoho Creator) for custom logic. The vendor largely controls the runtime, the data services and the way applications are hosted and scaled. This brings rapid prototyping, built-in sign-in and simple deployment, but it also means the team has less direct control over performance, architecture and portability as the application grows.

4. Case Scenario: A Hospital Network

4.1 Organisation and objective

Consider a fictional network of 12 hospitals that handles about 3 million appointments a year. Its old appointment and patient-records software has been patched for many years. Management decides to replace it with a low-code platform, extending a simple booking application like the one built in this lab, and plans to finish in 15 months at a lower cost than a conventional rewrite.

4.2 Complexity factors

Factor	Requirement in the Scenario
Volume	Tens of thousands of booking transactions per hour at peak times, such as vaccination drives.
Business rules	Many rules per department and doctor: slot lengths, referrals, insurance cover and priority cases.
Integration	Links to billing, laboratory, pharmacy, insurance and government health portals.
Compliance	Patient-data privacy laws, audit trails and requirements on where data is stored.

Factor	Requirement in the Scenario
Lifespan	An expected service life of 10 years or more with continuing changes.

Table 1: Complexity Factors in the Scenario

4.3 Outcome

The patient booking portal and a basic scheduling module were ready within three months. Progress then slowed when the team moved to insurance rules, lab and billing integration, and peak-load handling. Custom code and external services had to be added to work around platform limits, and the project finally settled on a hybrid design, keeping low-code for forms, workflows and dashboards. The final cost was higher than the original estimate for a conventional build.

5. Limitations Encountered

5.1 Scalability and performance

The platform's generated runtime and data layer could not be tuned as freely as hand-written code. Options such as query design, caching and indexing were limited to what the platform exposes, and scaling depended on the vendor's infrastructure and plan tier. Problems that would be fixed by refactoring in a conventional project became support requests or paid upgrades.

5.2 Complex business logic

Visual workflows and scripts suit straightforward rules, such as the past-date and clash checks in the Validate appointment workflow. Hundreds of interacting rules are harder to read, test and version in a visual tool, and automated testing and code review are less natural than in a code repository.

24140112

5.3 Integration with other systems

Standard connectors cover common cloud services, but older hospital systems and unusual protocols need custom adapters. The CRM integration attempted in this lab series was a small example: the built-in connection could not be authorised because of a plan limit, and a manual API route was needed instead. At enterprise scale such workarounds multiply and become harder to maintain.

5.4 Vendor lock-in

Applications depend on the vendor's own data model, scripting language and runtime. Moving away later can mean rebuilding the application, which raises switching costs and weakens the organisation's position when licences are renewed.

5.5 Customisation limits

Requirements outside the component library, such as unusual interface behaviour or specialised algorithms, need workarounds. These add technical debt and reduce the speed advantage that low-code was chosen for.

5.6 Security, compliance and data location

Built-in security features do not remove the need to check the underlying controls. Auditors may ask for evidence on encryption, patching and where data is stored, much of which depends on the vendor's documentation. Wrongly configured permissions by inexperienced builders are another risk, as discussed in Ex 12.

5.7 Governance and application sprawl

Because applications are easy to create, different teams may build overlapping tools with duplicate data and inconsistent security settings, which makes support and maintenance harder.

5.8 Cost at scale

Licences are often tied to users, applications or resource use, and costs can rise quickly as patient numbers and environments grow. In this lab, even a single extra connection hit a trial-plan limit, a small reminder that plan tiers shape what can be built.

5.9 Skills, testing and DevOps

Developers experienced in one platform can be scarce, and links to automated testing, source-control branching and deployment pipelines are often more limited than in conventional projects.

5.10 Summary of limitations

Limitation	Typical Symptom	Severity
Scalability	Not enough throughput at peak load	High
Complex logic	Hard-to-read models and limited testing	High
Integration	Fragile custom adapters and plan limits	High
Vendor lock-in	Costly exit and weaker pricing leverage	High
Customisation	Workarounds and technical debt	Medium
Security and compliance	Evidence depends on vendor information	Medium
Governance	Duplicate apps and uneven standards	Medium
Cost at scale	Licensing above forecast	High
Skills and DevOps	Scarce talent, limited pipeline options	Medium

2414011128

Table 2: Summary of Limitations

6. Impact Analysis

The figures below are illustrative for the fictional scenario and show how platform constraints can add up. They are not measured data.

Metric	Initial Plan	Illustrative Outcome
Delivery time	15 months	24 months
Cost vs. conventional build	30% lower	About 10% higher
Peak bookings per hour	40,000	About 15,000 before redesign
Share built on low-code	100%	About 50%, mainly forms and workflow
Custom code or external services	Minimal	Significant for core rules and integration

Table 3: Illustrative Plan versus Outcome

Beyond the numbers, repeated redesign can weaken stakeholder confidence and delay commitments. Early speed should therefore be judged together with lifecycle cost and operational needs.

7. Low-Code versus Traditional Development

Aspect	Low-Code Platform	Traditional Development
Initial speed	Fast for standard applications	Usually slower at the start
Complex logic	Can be hard to manage visually	Flexible and testable in code
Scalability control	Limited to vendor runtime and options	Greater architectural control
Integration	Convenient for common connectors	Custom integration is possible
Portability	Often low because of platform dependency	Higher with open standards
Cost profile	Low start-up cost; licences may grow	Higher upfront effort; costs easier to control
Skills	Platform-specific	Broader engineering skills
Best fit	Forms, workflows, dashboards, prototypes	Core systems and high-performance workloads

Table 4: Comparison of Approaches

8. Lessons and Recommendations

- **Test the hardest requirements first.** Use a proof of concept on peak volume, rule complexity and integration before committing.
- **Use a hybrid architecture.** Keep forms, dashboards and fast-changing workflows in low-code; build performance-critical logic as services exposed through APIs.
- **Plan for portability.** Document data models and business rules outside the platform and agree export rights.
- **Estimate total cost of ownership.** Include licences at expected scale, environments and workaround effort over several years.
- **Set governance early.** Define ownership, naming, security standards and approval steps for new applications.
- **Keep engineering discipline.** Use version control, testing and code review wherever the platform allows.
- **Verify security and compliance.** Ask for certifications and written commitments on data location.

2411401128

9. When Low-Code Is Still a Good Choice

Low-code remains a strong choice for internal approval processes, data-entry forms, dashboards, self-service portals, process automation, prototypes and applications of modest scale, which describes the Patient Appointment Booking application built in this lab series. The lesson is not to avoid low-code, but to avoid giving it the most demanding core workloads without first checking that it fits.

10. Conclusion

Low-code platforms speed up delivery and let more people take part in building applications, but their abstraction can limit scalability, flexibility, portability, cost control and governance in complex enterprise settings. A deliberate hybrid approach, with low-code where it adds clear value, conventional engineering where deeper control is needed, and governance and exit planning from the start, is usually the more reliable path.

11. References

- Bock, A. C., and Frank, U. (2021). “Low-Code Platform.” *Business & Information Systems Engineering*, 63(6), 733–740.
- Sahay, A., Indamutsa, A., Di Ruscio, D., and Pierantonio, A. (2020). “Supporting the understanding and comparison of low-code development platforms.” 46th Euromicro Conference on Software Engineering and Advanced Applications.
- Zoho Creator documentation: https://www.zoho.com/creator/help/
- Analyst reports on low-code application platforms from Gartner and Forrester.

Note: the organisation, scenario and outcome figures are illustrative and should not be read as verified industry measurements.

241401128